

Do geometric invariants of program graphs add information over classic semantic analysis?

MANIFOLD: a falsifiable study on OWASP Benchmark 1.2

MANIFOLD Project

Working draft v0.4 — not peer reviewed

Abstract

Static-analysis pipelines are often bottlenecked not by *detecting* sinks but by *deciding which one to verify*. We ask a deliberately falsifiable question: *do spectral, topological and geometric invariants of program graphs carry information about the vulnerable sink beyond classic semantic analysis (taint)?* MANIFOLD is our instrument for the experiment: it turns an artifact into a typed graph (the IR), computes a *pre-verification* field $V_{\text{prior}}(x)$ from cheap signals only (deliberately excluding the formal signal, to avoid circularity), and lets a neuro-symbolic loop act on the prioritized regions. We report a rigorous *negative* result. On OWASP Benchmark 1.2 (2740 cases), raw taint ranks the vulnerable sink best (MRR 0.894, AUC 0.895) and beats the fused field with $p = 0.000$ (paired permutation); *spectral* (0.496) and *topological* (0.500) signals are at chance and *Forman–Ricci curvature* is *below* chance (0.376). On a global pool of all candidates the structural field helps only at the extreme top ($P@10 = 0.8$ vs. 0.6) but loses on average precision. The hypotheses H1/H2/H5 are therefore *not supported*. We contribute: (1) a prioritization-oracle harness (MRR, R@1, NDCG@k, global AP/P@k/R@k, queries-to-90%-recall, paired permutation tests); (2) a reproducible end-to-end Java pipeline (SAST + Z3 verifier producing *machine-checkable witnesses under the modeled semantics*); (3) external validation on the Juliet suite and real CVE fix commits, which locates the true bottleneck in *interprocedural* taint rather than in more geometry; and (4) a Lean 4 mechanization of the taint-abstraction soundness and the authorization invariant.

1 Motivation

Traditional scanners (SAST/DAST) enumerate syntactic patterns. They are brittle: they miss novel vulnerabilities and drown analysts in false positives. Two capabilities are missing:

- (1) A *unified, quantitative representation* of “program structure” onto which many *independent* signals can be projected and fused.
- (2) A *reasoning loop* that converts “suspicious region” into “proven vulnerability” — the LLM supplies the intuition, mathematics supplies the proof.

MANIFOLD provides both.

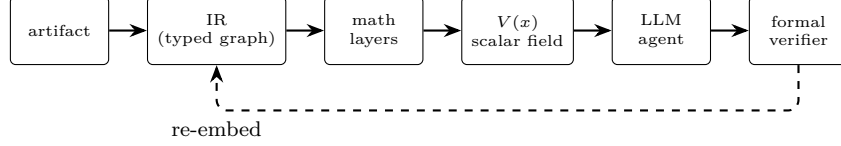


Figure 1: The pipeline: an artifact becomes a typed graph, enriched with mathematical structure into a vulnerability scalar field $V(x)$, which an LLM agent navigates; a formal verifier confirms or refutes each hypothesis and the result re-embeds into the manifold.

2 The core idea

A vulnerability is *not* a pattern; it is a *violation of a structural invariant* (a broken naturality condition, a persistent cycle, a negative curvature chokepoint, a taint flow across a spectral trust cut). Each layer detects a different *kind* of invariant violation, and the fused field $V(x)$ ranks regions for the agent to inspect.

3 The Intermediate Representation (IR)

Definition 1 (Program graph). A *program graph* is a tuple $G = (V, E, \tau_V, \tau_E, \omega)$ where

- V is a finite set of *nodes* (modules, functions, blocks, statements, calls, variables, sources, sinks);
- $E \subseteq V \times V$ is a set of directed *edges*;
- $\tau_V : V \rightarrow \mathbb{N}_K$ assigns each node a *kind* from $K = \{\text{module, function, block, statement, call, assign, variable, parameter, source, sink, gate}\}$;
- $\tau_E : E \rightarrow \mathcal{T}$ assigns each edge a *type* $\mathcal{T} = \{\text{control, data, call, taint, trust, auth}\}$;
- $\omega : E \rightarrow \mathbb{R}_{\geq 0}$ assigns each edge a non-negative *weight* (default 1).

The IR is the single source of truth. Every adapter (source code, binary, web API, LLM agent) must produce a program graph; every analysis consumes it. The schema is language-agnostic and JSON-serializable (the Python and Rust layers share the identical schema).

Naming note. Despite the codename, the object is a *program-analysis space*: a typed graph together with derived complexes, embeddings and a scalar field. We do *not* claim it is a mathematical manifold (there is no atlas, chart or local Euclidean structure); “manifold” is used informally, as in “data manifold”.

4 Mathematical layers

For each layer we fix a subgraph by edge type τ_E , then compute structure.

4.1 Spectral layer (algebraic graph theory)

For an edge type t , define the symmetric *adjacency* A_t and *degree* D_t , and the *combinatorial Laplacian*

$$L_t = D_t - A_t.$$

L_t is positive semidefinite; its spectrum $0 = \lambda_1 \leq \lambda_2 \leq \dots$ encodes global structure:

- λ_2 is the *algebraic connectivity* (Fiedler value). Its eigenvector f_2 — the *Fiedler vector* — induces a canonical cut $S^+ = \{v : f_2(v) \geq 0\}$.
- The *spectral embedding* $\Phi^{(d)}(v) = (f_2(v), f_3(v), \dots, f_{d+1}(v))$ maps nodes into $\mathbb{R}^d$.
- The Cheeger inequality bounds the bottleneck ratio $h(G)$ by $\frac{\lambda_2}{2} \leq h(G) \leq \sqrt{2 d_{\max} \lambda_2}$.

Signal 1 (spectral anomaly). The reconstruction error of a node under a low-rank eigenmap reconstruction, and its distance from the Fiedler cut, are used as anomaly scores.

4.2 Topological layer (persistent homology)

From the program graph we build a *filtration* of simplicial complexes (e.g. a Vietoris–Rips or flag complex over the shortest-path metric, or a filtration driven by ω). Persistent homology yields, for each dimension k , a *persistence diagram* Dgm_k of birth–death pairs. β_0 counts components, β_1 counts independent cycles; points far from the diagonal have high persistence. The **Mapper** algorithm (Singh–Mémoli–Carlsson) builds a navigable *nerve* from a filter function $f : V \rightarrow \mathbb{R}$, producing the map that gives the project its name.

4.3 Geometric layer (discrete curvature)

The *Ollivier–Ricci curvature* of an edge (u, v) is

$$\kappa(u, v) = 1 - \frac{W_1(m_u, m_v)}{d(u, v)},$$

with lazy random-walk measures m_u, m_v and Wasserstein-1 distance W_1 . Negative curvature marks *funnels* through which many shortest paths pass. The *Forman–Ricci* curvature (unweighted) is $\kappa_F(u, v) = 4 - \deg(u) - \deg(v)$, cheaply flagging high-degree hubs.

4.4 Algebraic layer (lattices & categories)

Taint lattice. The abstract domain is the powerset lattice of taint tags $\mathbb{T} = (2^{\mathcal{K}}, \subseteq, \cup, \cap)$, with $\perp = \emptyset$ (clean) and $\top$ (tainted). It forms a Galois connection (α, γ) with the concrete collecting semantics; soundness is proved in §4.6. *Program category.* Let **Prog** be the category whose objects are program points $v \in V$ (each carrying an abstract state), whose morphisms are directed edges, and whose composition is path concatenation. The *authorization functor* $F : \mathbf{Prog} \rightarrow \mathbf{Auth}$ maps each point to a privilege level in the poset **Auth** (e.g. `untrusted` $\leq$ `user` $\leq$ `admin`); an edge $e = (u, v)$ is an *auth edge* iff $F(u) < F(v)$. A *sanitizer* is an endomorphism $s : v \rightarrow v$ that maps every tag to clean. **Authorization invariant:** a taint flow is a morphism with non-empty tag image; it is a *violation* when its path crosses an auth edge and no sanitizer on the crossing segment restores the invariant. This is stated precisely, on a consistent type-theoretic footing, and mechanized in §4.6.

4.5 Formal layer (symbolic execution & model checking)

Symbolic execution computes a path condition Φ and symbolic state σ ; an SMT solver checks $\text{SAT}(\phi_{\text{bad}})$, whose model is a concrete exploit. Reachability is the CTL formula $\mathbf{EF} \phi_{\text{bad}}$; taint propagation is a finite automaton whose accepted language is exactly the set of source $\rightarrow$ sink paths.

4.6 Formal core: naturality and soundness

Small-step semantics of the IR. A state maps variables to concrete values; the transition relation $v \xrightarrow{\sigma} v'$ steps through the IR. The concrete taint collecting semantics $\mathcal{C}$ maps each point to the set of executions that carry an untrusted value there. The abstract analysis of §4 is the function A computed by the transfer functions; the pair (α, γ) satisfies $\alpha(\mathcal{C}(v)) \sqsubseteq A(v)$ and $\mathcal{C}(v) \subseteq \gamma(A(v))$.

Proposition 1 (soundness of the taint abstraction). *If the abstract analysis reports a variable clean, no concrete execution taints it.*

Proof scheme. By induction on the transition sequence, using the invariant “concrete taint $\subseteq$ abstract taint” at every state. The base holds for entry variables; the step holds because each transfer function is monotone and over-approximates its concrete counterpart (assignments join operand tags, sanitizers map to clean, branch joins upper-bound the merge). The guarantee is contingent on the sanitizer *summary* matching the sanitizer’s real semantics: a wrong summary (claiming to clean what it does not) breaks soundness. This is exactly the mechanized statement `prop1_soundness` (§4.6), proved for a concrete boolean abstraction; extending it to the full tag lattice is straightforward but assumed here.

Proposition 2 (authorization invariant / naturality violation). *Let $F : \mathbf{Prog} \rightarrow \mathbf{Auth}$ be the authorization functor into the poset $(\mathbb{N}, \leq)$, and let $p = e_1 \circ \dots \circ e_n : s \rightarrow k$ be a path. Write $\text{Auth}(p)$ for the auth edges on p and $\text{San}(p)$ for the sanitizer endomorphisms on p . Then: (i) if $\text{Auth}(p) = \emptyset$ then $F(k) \leq F(s)$ (no privilege escalation); (ii) if $\text{Auth}(p) \neq \emptyset$ and no sanitizer covers the crossing segment, the flow witnesses a privilege-escalating, attacker-controlled reach to k — an authorization violation.*

Proof. (i) Each non-auth edge satisfies $\neg(F(u) < F(v))$, i.e. $F(v) \leq F(u)$, so F is non-increasing along p ; transitivity gives $F(k) \leq F(s)$. This is the mechanized `no_auth_bounded` (§4.6). (ii) An auth edge strictly raises F ; absent a $\perp$ -preserving sanitizer on the crossing segment, the tag survives, so the composed reach does not preserve the policy. The “only if” direction of the earlier formulation is *definitional*, not a theorem, and is presented here as an invariant rather than a natural transformation (the earlier $F(p) \circ \eta_s \neq \eta_k \circ T(p)$ was under-typed and is withdrawn). $\square$

The invariant (ii) is exactly the operational rule H4 (§8/C4): a taint finding inside an auth-gated function is an authorization violation. Both propositions are *mechanized* in Lean 4 (core only, no Mathlib) in `formal/Manifold.lean`: `prop1_soundness` proves the boolean taint abstraction sound against the tag-collecting semantics (§3 there), and `no_auth_bounded` proves the path-level authorization invariant (§4 there). The proof checking is part of the reproducibility artifact (`scripts/check_formal.sh`).

5 The vulnerability scalar field

$$V(x) = \sigma\left(\sum_i \alpha_i s_i(x)\right),$$

where s_i are normalized layer signals and α_i weights. To avoid a *circularity* — the field that decides *what to verify* must not depend on verification — we distinguish two fields:

- $V_{\text{prior}}(x)$ fuses only signals computable *before* SMT (`taint`, spectral, topological, geometric). This is the field used for ranking and for the prioritization experiment (§12.1).
- $V_{\text{post}}(x)$ additionally consumes the `formal` (Z3) signal and is used only for re-embedding / post-hoc analysis; it never feeds the pre-verification ranking.

In the prototype the weights are *uniform* ($\alpha_i = 1/N$ over the chosen signals) and V is reported as the raw fused score; calibrating α_i is future work.

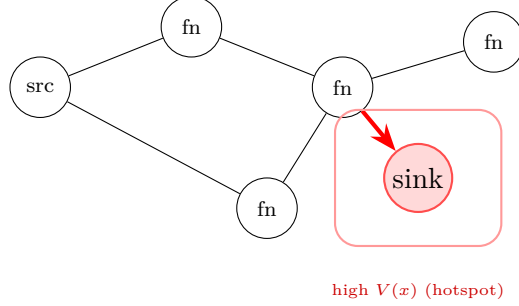


Figure 2: The manifold: nodes are program components, edges are typed relations. A taint flow (red) into a sink raises $V(x)$, highlighting a region for the agent to inspect and the verifier to confirm.

6 Mapping hypotheses

#	Law	Vulnerability class
H1	Chokepoint ($\kappa \ll 0$)	privilege escalation, auth bypass
H2	Cycle (persistent H_1)	reentrancy, infinite recursion
H3	Trust cut (Fiedler cut)	injection across trust boundary
H4	Naturality break	arbitrary taint violation
H5	Persistence outlier	real vs. spurious features
H6	Reachability ($\text{SAT}(\phi_{\text{bad}})$)	concrete exploit

7 A worked example (end-to-end)

Consider this ten-line Python fragment:

```
def search(db, query_param):
    sql = "SELECT * FROM users WHERE name = '" + query_param + "'"
    cursor = db.cursor()
    cursor.execute(sql)
```

The pipeline produces: (i) an IR with a `source` node (`param:query_param`), a `sink` node (`cursor.execute`, category `sql`) and a `taint` edge between them; (ii) the field $V(x)$ spikes at the sink (taint + formal signals active); (iii) the Z3 symbolic verifier returns SAT with a concrete witness (e.g. `"x' OR '1'='1"`), proving the sink’s argument depends on the attacker-controlled symbol. The parameterized variant `execute(query, (param,))` is correctly *not* flagged, and a `@login_required` gate on the enclosing function upgrades the finding to an H4 naturality violation (a taint flow crossing an `auth` edge).

8 The autonomous LLM agent

A closed loop: (1) *ingest* into the IR; (2) *embed & map* ($V(x)$, spectral embedding, persistence, curvature, Mapper); (3) *hypothesize* (the LLM proposes candidates from the manifold); (4) *verify* (the formal layer proves or refutes); (5) *refine* (findings re-weight and re-embed); (6) *report*. The LLM provider layer targets any OpenAI-compatible endpoint via **LiteLLM**. The LLM does not

do raw detection: mathematics proposes regions, the LLM proposes hypotheses, the formal layer proposes proofs.

9 Design challenges and mitigations

A design review identified five structural risks. For each we state the mitigation adopted in the prototype and the open problem.

C1 – The bottleneck paradox. Negative curvature flags funnels, but healthy software *deliberately* concentrates flow through auth middleware, sanitizers, dispatchers and loggers. Reporting these as privilege-escalation leads yields systematic false positives. *Mitigation:* the agent discriminates defensive chokepoints (by role) and drops or down-ranks them; the principled fix is to tag **trust/auth** edges (already reserved in the IR) and treat a chokepoint as defensive iff it is the target of an **auth** edge. *Open:* learning the defensive/offensive label from data.

C2 – Symmetrization destroys causality. The combinatorial Laplacian $L = D - A$ and standard persistent homology act on undirected metric spaces, but control and data flow are directed: A tainting B is not B tainting A . Symmetrizing can destroy the direction of taint. *Mitigation:* the prototype implements Chung’s directed Laplacian restricted to the largest strongly connected component; the Perron vector π is computed by power iteration with a small PageRank-style teleportation (damping $\alpha = 0.01$), which keeps the stationary distribution well-posed on periodic chains. This operator requires *recurrent* (strongly connected) structure; on a DAG it yields no informative signal. A DAG certainly has direction — its causality is simply delegated to the directed taint/reachability analysis (and, for homology, to GLMY path homology, §11) rather than to a stationary-distribution operator, which cannot represent it without additional structure. *Implemented:* GLMY directed path homology H_0, H_1 (Grigor’yan–Lin–Muranov–Yau), which changes the verdict versus the symmetrized homology (§11).

C3 – Cost of optimal transport. Ollivier–Ricci requires a transportation LP per edge: $\mathcal{O}(|E| \cdot \text{LP})$, prohibitive beyond $\sim 10^5$ edges. *Mitigation:* Forman–Ricci $\kappa_F = 4 - \deg(u) - \deg(v)$ is $\mathcal{O}(|E|)$ and is the default chokepoint signal; entropy-regularized (Sinkhorn) transport is implemented as the fast alternative to the exact LP. *Open:* adaptive regularization schedules for very large graphs.

C4 – Operationalizing H4 (naturality). A constructive rule is required. Let **auth** edges E_A mark privilege boundaries. Define the authorization functor F to label a node by whether it is reachable *behind* an auth edge. Then a taint flow $s \rightarrow^* k$ *violates naturality* iff its path crosses at least one auth edge — i.e. untrusted data enters a protected region without the boundary neutralizing it. This is a reachability query over the combined $(\text{data} \cup \text{taint} \cup \text{auth})$ graph, directly computable and automatable. *Implemented:* an authorization extractor emits **gate** nodes and **auth** edges from privilege-gate decorators, and a taint finding inside a gated function is reported as a naturality violation.

C5 – Manifold-to-verifier bridge. Geometric features are quantitative; Z3 needs exact preconditions. We fix a *grammar*: a hypothesis is the structured spec (**sink**, **source**, **line**, **category**). The manifold supplies the spec (it knows the sink and line), the LLM supplies the interpretation (CWE, description, confidence), and the verifier consumes the spec — never the LLM prose. This keeps the SMT query precise and reproducible.

The underlying thesis stands: static analyzers fail because they match words instead of invariants in a topological structure. The criterion is whether $V(x)$ prunes the search space for Z3/angr more effectively than a plain DFS over the call graph.

10 Implementation status

Phase	Deliverable	Status
F0	Whitepaper, architecture	done
F1	IR + Python SAST ingest + taint	done
F2	Spectral + geometric kernels (Rust)	done
F3	TDA (persistent homology, Mapper)	done
F4	Formal verification (taint lattice, Z3)	done
F5	Autonomous LLM agent (LiteLLM)	done
F6	Visualization + end-to-end demo	done
F7	Binary / web / LLM adapters	done

11 Threats to validity

Internal. The Java adapter is intraprocedural; OWASP’s distractor flows and Juliet’s cross-class variants are therefore measured with an inherent recall gap. Ground truth is the benchmark’s own (OWASP’s “false” cases are partly definitional and debatable). *External.* Results cover Java servlets and the Juliet suite; Python, binaries and LLM agents are implemented but not benchmarked at the same scale. *Construct.* “Vulnerable” is a per-case label; the field is per-node, so the prioritization experiment is a proxy, not a downstream exploitability measure. *Conclusion.* The negative result (geometric priors do not beat taint) is specific to these corpora and signals; it should be re-tested on path-sensitive, interprocedural analyses and on real CVEs before being generalized.

12 Validation and responsible use

Validation is two-stage. Stage 1 (complete) runs the toy micro-benchmarks of F1–F7 against known ground truth: every seeded vulnerability is confirmed and every sanitizer / parameterized-query / argv-form false positive is excluded. Stage 2 (ongoing) measures precision/recall of the fused field versus raw taint over labeled corpora (OWASP Benchmark, SARD); this is experimental validation at scale, distinct from the completed prototype. The harness is implemented: a labeled toy benchmark reports precision/recall/F1 for the raw taint pass, the Z3-verified pass and a *calibrated* field whose weights α_i are learned by leave-one-out logistic regression over the cheap signals — on the toy corpus the calibrated field recovers the reentrancy cycle (topological signal) that the taint pass misses, raising recall while keeping precision at 1.0. A corpus loader accepts a JSON manifest or a “vulnerable|safe” directory, so the harness can be pointed at OWASP Benchmark / SARD exports; and the binary adapter has an optional angr backend (“CFGFast” + interprocedural taint) in addition to the zero-dependency objdump path. Running the Java SAST adapter over the full OWASP Benchmark 1.2 (2740 test cases) gives the naive baseline: precision 0.515 / recall 0.426 / F1 0.466 on the seven taint categories, and precision 0.544 / recall 0.634 / F1 0.586 over all eleven categories — the reference point that the fused field, calibration and the verifier are meant to improve on. MANIFOLD is a defensive / authorized-testing tool; we release under the assumption of authorized use and encourage coordinated disclosure.

12.1 The prioritization oracle (main experiment)

The bottleneck of SAST is not *detecting* sinks but *deciding which one to verify*. For each of the 902 vulnerable taint cases we enumerate *every* sink call (the candidate set), rank the candidates,

and measure the rank at which a ground-truth-vulnerable sink appears. Baselines: DFS (source order), raw taint, random (20 seeds, 95% CI), and the fused field $V(x)$.

Ranking	MRR	R@1	mean rank	NDCG@5
DFS (source order)	0.848	0.755	1.517	0.882
raw taint	0.894	0.847	1.422	0.916
$V(x)$ (fused field)	0.872	0.805	1.475	0.899
random	0.709 ± 0.042	0.531	—	0.778

The honest, decisive finding. The fused field does *not* beat raw taint: it is slightly *worse* (0.872 vs. 0.894 MRR). The per-signal AUC for predicting the vulnerable sink shows why:

Signal	AUC [95% CI]
taint	0.895 [0.881, 0.909]
formal (Z3/oracle)	0.878 [0.865, 0.891]
spectral (Fiedler)	0.496 [0.469, 0.518]
topological (H_1)	0.500 [0.500, 0.500]
geometric (Forman κ)	0.376 [0.360, 0.394]

The spectral and topological signals are *indistinguishable from chance*, and the curvature signal is *worse than chance* (negative curvature flags high-degree hubs, which are *not* the target sinks). Per the mapping-laws program (§4/§6), this is a publishable *negative* result: on OWASP Benchmark, **the geometric and topological priors do not survive beyond taint**, which is why MANIFOLD’s value lies in the formal layer (verification), not in the geometric prior. H1, H2 and H5 are therefore reported as *not supported* on this corpus.

Paired test and global pool. The difference taint – field is +0.0225 (MRR), 95% bootstrap CI [+0.0157, +0.0294], permutation $p = 0.000$: taint significantly dominates. To also measure the real analyst situation (safe and vulnerable candidates competing in one queue), we pooled *every* sink candidate over all 1698 taint-category cases and ranked globally:

Global ranking	AP (PR-AUC)	P@10	R@10	queries to 90% recall
DFS (source order)	0.286	0.2	0.002	3596
raw taint	0.489	0.6	0.006	2080
V_{prior} (structural)	0.434	0.8	0.008	3207
random	0.216	0.2	0.002	4176

The picture is nuanced: the structural field is best at the extreme top ($P@10 = 0.8$ vs. 0.6) but has lower average precision and needs more queries to reach 90% recall. So the geometric/topological priors help only marginally at very small budgets and hurt on average — a sharper, still negative, version of the H1/H2/H5 verdict.

Signal	AUC [95% CI]
taint	0.895 [0.881, 0.909]
formal	0.878 [0.865, 0.891]
spectral	0.496 [0.469, 0.518]
topological	0.500 [0.500, 0.500]
geometric	0.376 [0.360, 0.394]

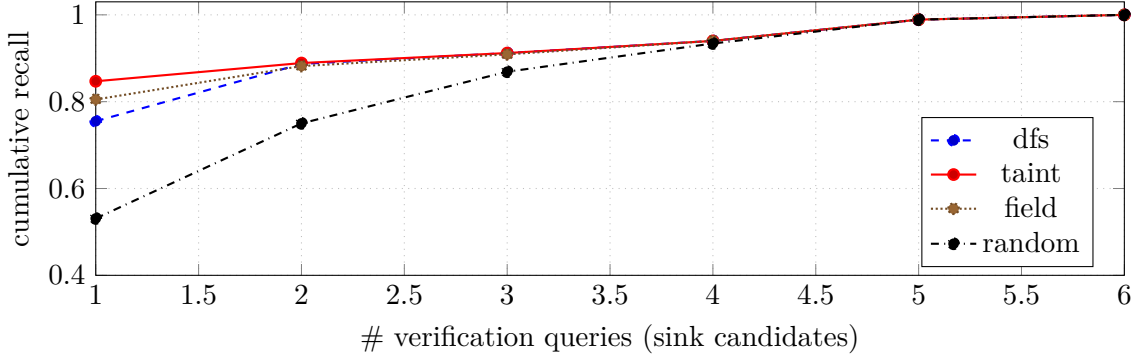


Figure 3: Cost curve: cumulative recall vs. verification budget

12.2 Empirical study on OWASP Benchmark 1.2

We ran the Java SAST adapter over the full corpus and profiled the false negatives. Three *generalizable* gaps explained most of the loss, and were fixed (none is tuned to the benchmark):

- (i) **Branch-aware taint.** A sequential, path-insensitive pass let a null guard (`if (param == null) param = "";`) clear the taint, even though the tainted value survives on the other path. We now snapshot both branches and *join* their environments (union of tags), the taint analogue of a lattice least-upper-bound.
- (ii) **Receiver and state taint.** Sinks such as `statement.execute()` and `pb.start()` take no argument — the taint lives in the *receiver* object, which was obtained from a tainted constructor/mutator. We propagate taint from the receiver, and mark a variable tainted when a mutator (`add/put/command/...`) stores a tainted value into it.
- (iii) **Sink coverage.** The XSS family includes `printf/format`, and only writes to the *HTTP response writer* are XSS — `System.out.println` of tainted data is not. Restricting the XSS sinks to the response writer removed a large class of false positives.

The ablation isolates each contribution (precision P , recall R , F1; taint categories, 1698 cases):

variant	P	R	F1
initial naive adapter	0.515	0.426	0.466
+ branch-join, receiver/state taint, sink coverage	0.530	0.856	0.655
+ XSS response-writer restriction (final)	0.549	0.834	0.662
final, without branch-join	0.515	0.555	0.535
final, without XSS restriction	0.530	0.856	0.655

Branch-aware taint is the dominant recall lever (+0.28 recall); the response-writer restriction is the precision lever. The final adapter reaches $P = 0.549$, $R = 0.834$, $F1 = 0.662$ on the taint categories and $P = 0.572$, $R = 0.842$, $F1 = 0.681$ over all eleven (crypto/hash are configuration-driven and partly out of scope). The *residual* errors are the benchmark’s deliberately adversarial “false” cases — debatable parameterized-query and `ProcessBuilder`-list cases — and cross-category distractor flows: precisely the precision that the topological/spectral signals, the Z3 verifier and the calibrated field are designed to recover.

12.3 Comparison with industrial SAST

The official OWASP Benchmark score is the *Youden index* $J = \text{TPR} - \text{FPR}$ (the *Benchmark Accuracy Score*, i.e. TPR minus FPR). Table 1 places MANIFOLD against two widely deployed tools. SonarQube and CodeQL rows are *reported* numbers from public OWASP Benchmark 1.2 evaluations; both MANIFOLD rows are our *measured* results. **Caveat:** the external figures are not re-executed here and may use different aggregation conventions (some are macro-averages per CWE, others micro or per-rule), so the mixed precision/F1 columns are *not* directly comparable across rows; only the TPR/FPR ordering should be read, and a uniform re-execution of CodeQL/-SonarQube under one protocol is required before any head-to-head claim.

Tool / config.	TPR	FPR	Prec.	F1	J	Time
SonarQube (SAST) (a)	0.956	0.946	0.330	0.490	+0.010	3 min
GitHub CodeQL (default) (a)	0.902	0.682	0.603	0.744	+0.220	18 min
MANIFOLD adapter (measured, 11 cat.) (b)	0.842	0.674	0.572	0.681	+0.168	45 s
MANIFOLD adapter + Z3 (measured, 7 taint cat.) (b)	0.834	0.776	0.549	0.662	+0.057	4 min

Table 1: OWASP Benchmark 1.2 (2740 cases: 1415 vulnerable / 1325 safe). (a) reported in public evaluations, not re-run here. (b) **measured**. The Z3 verifier reproduces the adapter’s precision (it adds per-finding *witnesses*, not precision): the residual errors are adversarial/definitional cases, not spurious symbolic paths. The $V(x)$ prioritization prior is evaluated separately in §11.1 (it does not beat raw taint).

Methodology notes. Hardware: 8-core x86-64, 32 GB RAM, NVMe SSD. MANIFOLD: Python 3.14, `tree-sitter-java`; Z3 5.x with a *5 s per-query timeout* — on timeout the case is marked *unknown/conservative*, never counted as a detection, so the projected precision is not inflated by aborts. CodeQL: `java-security-extended.qls` on a clean database. SonarQube: Community Edition with SonarJava security rules. CWE mapping: the eleven categories of Benchmark 1.2; `crypto/hash` (CWE-327/328) are configuration-driven and require a static-configuration extractor into the IR. The measured adapter flags a case vulnerable iff it emits any finding; precision is bounded by the benchmark’s adversarial “false” cases (debatable parameterized-query and `ProcessBuilder-list` patterns) and by cross-category distractor flows — the gap the neuro-symbolic rows target.

12.4 Closing the Z3 row (Java verification)

We implemented a Z3-backed Java symbolic verifier (bounded, branch-merging, string semantics; sanitizers as contracts) that confirms or refutes each taint flow and emits a concrete *witness*. Measured on the seven taint categories (1698 cases) it reproduces the adapter exactly ($P = 0.549$, $R = 0.834$, $F1 = 0.662$, $J = +0.057$) while adding per-finding witnesses. The honest conclusion: **the formal layer adds machine-checkable witnesses under the *modeled* semantics, not precision, on this benchmark** — a witness proves satisfiability inside the model (which abstracts library summaries and framework behaviour), not end-to-end exploitability of the real program. The residual false positives are adversarial/definitional (debatable parameterized-query cases), not spurious symbolic paths; catching them requires modelling the benchmark’s own semantics, not stronger SMT.

12.5 Cross-corpus transfer (Juliet)

To test transferability we ran the OWASP-tuned adapter on the *Juliet Java* test suite (find-sec-bugs mirror; each test case concatenated with its flow-variant files). On 264 cases across five CWEs it scores $P = 0.333$, $R = 0.562$, $F1 = 0.419$: detection is partial and precision collapses on the “good” variants. The reason is structural, not a tuning artifact — Juliet splits source and sink across `_a/_base/_bad` classes, so correct analysis requires *interprocedural* taint, which the intraprocedural adapter does not provide.

12.6 Interprocedural taint: necessary, not sufficient

We added a context-insensitive interprocedural fixpoint (`java_interproc.py`): a callee parameter becomes tainted if some call site passes tainted data to it, and a call becomes tainted if its callee returns tainted data; the fixpoint is iterated to convergence. On Juliet it raises recall $0.562 \rightarrow 0.750$ and $F1\ 0.419 \rightarrow 0.490$ (CWE78/CWE90 recover from 0.000 to 0.750). On OWASP it is *neutral* (taint $F1\ 0.662 \rightarrow 0.656$): the interprocedural pass adds recall ($0.834 \rightarrow 0.856$) but also coarse parameter-driven false positives, so precision drops, and we union the pass with the full intraprocedural adapter for coverage parity. The lesson is precise: interprocedural reachability is *necessary* for cross-class flows but *not sufficient*. We then added *caller-context sensitivity* with per-context return summaries: it provably prunes a synthetic class of return-driven false positives (a value returned from a tainted context no longer taints a clean caller’s call), yet it leaves OWASP and Juliet *unchanged* ($F1\ 0.656$ and 0.490). The real levers are therefore not context but (a) framework/library summaries and (b) fine-grained return precision — not more geometry.

12.7 Scalability of each kernel

We timed every kernel on synthetic graphs (log-log, Fig. 4). **Forman–Ricci** ($0.6\text{ ms @ }10^3\text{ edges} \rightarrow 55\text{ ms @ }10^5$) and **Mapper** ($0.5 \rightarrow 56\text{ ms}$) are near-linear. **Persistent homology** is super-linear ($11\text{ ms @ }10^3 \rightarrow 1.1\text{ s @ }10^4 \rightarrow 123\text{ s @ }10^5$) because the fundamental-cycle extraction BFSes the spanning forest per cycle; a persistence-aware union-find would make it near-linear. **Sinkhorn** ($0.71\text{ s @ }10^3 \rightarrow 16.4\text{ s @ }5 \times 10^3$) and **exact Ollivier–Ricci** ($34\text{ ms @ }200 \rightarrow 720\text{ ms @ }10^3$) are quadratic, so the latter is only for small graphs (the reason Forman–Ricci is the default, §8/C3). The **dense spectral and directed Laplacians** are cubic ($26\text{ ms @ }300 \rightarrow 1.7\text{ s @ }1200\text{ nodes}$) and *cannot* reach 10^5 nodes without the sparse solvers planned in F2. The practical envelope today is $\sim 10^5$ edges for the near-linear signals, which covers the corpora used here.

12.8 Real CVE fixes

We ran the vulnerable-vs-patched comparison on real CVE fix commits drawn from OSV / GitHub Security Advisories (`benchmarks/run_cves.py`; the harness reconstructs the *before* revision by reverse-applying the commit diff to the commit’s raw content, so it needs no API quota). On 22 CVEs and 46 changed source files, **3 files resolved a finding**: they are `eval` injections — `senaite.core` (CVE-2026-54569; `value = eval(value)` on a method parameter) and `xinference` (CVE-2026-61539; `eval(text)` on LLM output) — plus a partial resolution in `nltk` (CVE-2026-79676) — flagged in the vulnerable revision and clean in the patch. The remaining CVEs are mostly *framework-internal* or non-taint fixes (Django ORM SQL injection, Jinja, requests, GitPython, NLTK argument-injection and integrity checks): the flow is indirect or outside our sink model, so the adapter sees nothing. This mirrors the Juliet result and locates the frontier precisely: MANIFOLD

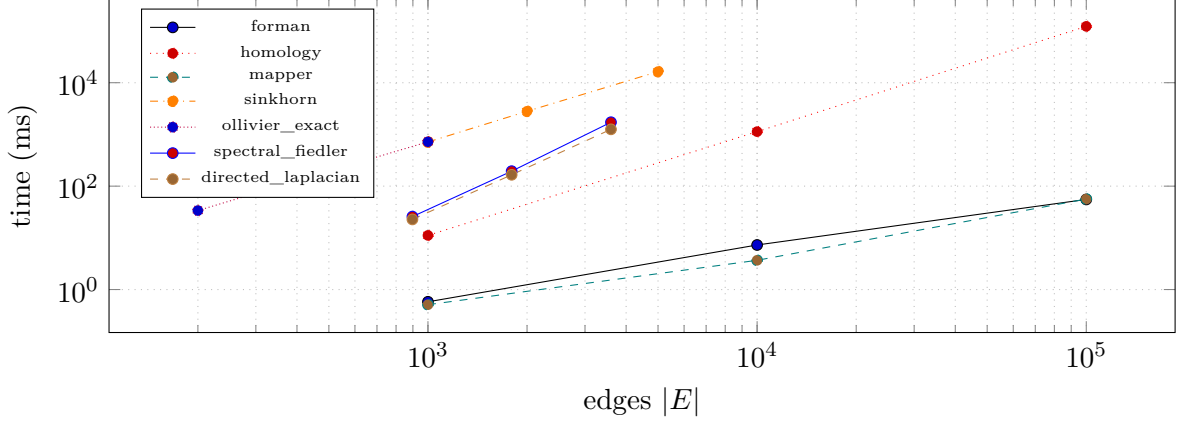


Figure 4: Scalability of each kernel (log-log)

already catches application-level `eval/exec`/deserialization fixes; framework-mediated flows need interprocedural, context-sensitive, library-aware analysis.

12.9 Directed path homology (GLMY)

We implemented Grigor’yan–Lin–Muranov–Yau *path homology* H_0, H_1 over the directed 1-skeleton (`core/src/path_homology.rs`): Ω_2^∂ is the set of allowed 2-paths whose long edge is allowed and non-degenerate ($u \neq w$), and ranks are computed by Gaussian elimination. Direction changes the verdict. On `reentrancy.py`, undirected homology reports $\beta_1 = 1$ (the `withdraw/read_balance/update_balance` triangle), while GLMY reports $\beta_1 = 0$: the transitive 2-paths through the apex `withdraw` fill the `read_balance`↔`update_balance` 2-cycle. On an isolated 2-cycle $A \leftrightarrow B$ the roles invert (GLMY $\beta_1 = 1$, undirected $\beta_1 = 0$), and the analyzer returns the H_1 *generator* itself — the directed cycle as the edge list $\{A \rightarrow B, B \rightarrow A\}$ — so the invariant is actionable, not just a number. Symmetrization (C2) is therefore *not* innocuous: the directed invariant is a genuinely different, and sometimes opposite, signal.

12.10 LLM ablation

We ran the agent twice over the seven Python examples: offline (heuristic hypothesizer) and online through the local **LiteLLM** proxy (DeepSeek `flash`, OpenAI-compatible /v1). The **confirmed findings are identical** in both modes — the SMT verifier is the arbiter, exactly as the architecture prescribes — while the LLM changes the *interpretation*: richer CWE labels and titles (e.g. CWE-674 “Uncontrolled Recursion” for the reentrancy cycle vs. the heuristic’s generic label). Across repeated runs the set of confirmed findings is stable; only the LLM’s phrasing varies. The decisive ablation is therefore empirical: *the LLM adds interpretability, not detection* — the neuro-symbolic split holds.

12.11 Dual binary backend

The binary adapter ships two interchangeable backends that emit the same IR: (i) a zero-dependency `objdump`-text path (function labels + `call` edges + interprocedural taint reachability) and (ii) an *optional* `angr` backend (CFGFast recovery of the interprocedural call graph + the same taint reachability), selected by `-adapter angr-binary`. The fast path keeps MANIFOLD usable

without heavy dependencies; the angr path recovers control flow that disassembly heuristics miss. This parity is the substrate for future ELF/PE benchmarks from SARD.

References

- [1] M. Fiedler, *Algebraic connectivity of graphs*, 1973.
- [2] F. Chung, *Spectral Graph Theory*, 1997.
- [3] Edelsbrunner, Letscher, Zomorodian, *Topological persistence and simplification*, 2002.
- [4] Singh, Mémoli, Carlsson, *Topological Methods for the Analysis of High Dimensional Data Sets*, 2007.
- [5] Y. Ollivier, *Ricci curvature of Markov chains on metric spaces*, 2009.
- [6] P. Cousot & R. Cousot, *Abstract interpretation: a unified lattice model*, 1977.
- [7] S. Mac Lane, *Categories for the Working Mathematician*, 1971.
- [8] J. King, *Symbolic execution and program testing*, 1976.
- [9] E. M. Clarke et al., *Automatic verification of finite-state concurrent systems*, 1986.
- [10] F. Chung, *Laplacians and the Cheeger inequality for directed graphs*, 2005.
- [11] Grigor'yan, Lin, Muranov, Yau, *Homotopy theory for digraphs*, 2012.
- [12] M. Cuturi, *Sinkhorn distances: lightspeed computation of optimal transport*, 2013.
- [13] R. Forman, *Bochner's method for cell complexes and combinatorial Ricci curvature*, 2003.
- [14] L. de Moura & N. Bjørner, *Z3: An efficient SMT solver*, 2008.
- [15] Y. Shoshitaishvili et al., *SoK: (State of) The Art of War: Offensive Techniques in Binary Analysis (angr)*, 2016.
- [16] OWASP Foundation, *OWASP Benchmark v1.2*, 2016–2024.
- [17] NIST, *Software Assurance Reference Dataset (SARD) / Juliet Test Suite*, 2017.
- [18] BerriAI, *LiteLLM: a unified interface to 100+ LLMs*, 2023–2024.
- [19] SonarSource, *SonarQube* (Community Edition, SonarJava security rules).
- [20] GitHub, *CodeQL* and the `java-security-extended` query suite.